

Falling with Style: Factoring up to 255 “with” a Quantum Computer

Craig Gidney

April 1, 2025

Abstract

In this paper, I explain how I factored all numbers up to 255 using Shor’s algorithm on a real quantum computer. I performed *exactly* the classical preprocessing specified by Shor’s algorithm, *exactly* the quantum circuit requested by Shor’s algorithm, and *exactly* the post-processing specified by Shor’s algorithm. In total this involved sampling the quantum computer 121 times. This slightly underperformed the 120 samples used when sampling from a random number generator instead.

1 Intro

Historically, most papers that claimed they “ran Shor’s algorithm” didn’t run Shor’s algorithm. It’s unfortunately common to run circuits *inspired by* Shor’s algorithm, but with key pieces replaced by trivial pieces [Van+01; Lan+07; Lu+07; Mar+12; Luc+12]. The issue is that, in the large shenanigans limit, all quantum factoring circuits are trivial [SSV13]. In this paper, I don’t make that mistake. I run Shor’s algorithm exactly as it was supposed to be run: with comically underoptimized circuits. Nevertheless, the circuits work. They quickly result in factors being produced. This naturally raises the question: “What’s the catch?”. Because of course there’s a catch.

The key insight here is that, for small numbers, Shor’s algorithm is surprisingly resilient to noise. It’s so resilient that, even if you replace the quantum computer with a random number generator, the algorithm still succeeds with high probability! This is serendipitous because, when a quantum computer is given a circuit that’s way too large, the output approximates a random number generator. In other words, for small numbers, Shor’s algorithm succeeds quickly *regardless of how well your quantum computer works*.

Factoring in this way reminds me of a scene from Toy Story. In the scene, Buzz Lightyear (see [Figure 1](#)) attempts to fly. He doesn’t actually fly but, due a humorous series of coincidences, appears to. Another toy, Woody, complains that Buzz was just “falling with style”. The rest of the toys don’t care. Similarly, in this paper, I will appear to succeed at factoring by falling with style.

2 Methods

I wrote python code (available at github.com/strilanc/falling-with-style) that produces a quantum circuit that performs the quantum part of Shor’s algorithm. The generated circuit performs a modular exponentiation and then a frequency basis measurement. I implemented the modular exponentiation by decomposing it into modular multiplications, then decomposing those into fused modular multiply-adds, then decomposing those into modular additions, then decomposing those into 2s-complement additions / subtractions / comparisons, then decomposing those into textbook CCX/CX/X gates [Cuc+04]. For the frequency basis measurement I wanted to use qubit recycling [ME99] to save space, but the quantum computer I chose (IBM_SHERBROOKE) doesn’t support classical feedback. So I simply performed the textbook quantum Fourier transform circuit followed by computational basis measurement. The textbook gates were converted into physical gates by Qiskit’s automatic transpilation tools.

I was initially worried that IBM’s quantum service would reject my factoring circuits, because IBM’s documentation only guarantees circuits will fit into hardware memory limits if they have fewer than ten

thousand two-qubit gates [IBM]. My circuits are much larger than this. For example, after transpilation, my circuit for factoring 15 weighs in at 44405 two-qubit gates. And my circuit for factoring 253 weighs in at 245750 two-qubit gates. Amazingly, despite the fact that they vastly exceed the allowed size, the system accepted these ridiculous circuits.

When I submitted the circuits, the two qubit gate error rate reported by the system was 1.41%. With this error rate, it would be difficult to extract signal from a computation that used a thousand gates nevermind a hundred thousand. So I was confident that my circuits would sufficiently randomize the output, achieving the goal of factoring by falling with style.

To insert a bit of competition, I decided to run my circuits not just against a real quantum computer but also against a simulated noiseless quantum computer and against a random number generator. I wanted to see how the real machine compared to these two extremes. To compare them, I counted how many times the quantum part of Shor’s algorithm was executed (how many quantum samples were collected). Fewer samples is better, because it’s more efficient.

Before showing the results, to ensure the reader understands what it means for a quantum sample to occur, I’ll quickly review the classical and quantum steps of Shor’s algorithm. Before talking to a quantum computer, Shor’s algorithm performs some classical preprocessing. First, it checks if n (the number to factor) is even, because even numbers would cause trouble later. If so, it succeeds by returning the factor 2. Second, it checks if n is prime. Prime numbers can’t be factored, so in this case the method returns an error saying no factor exists. Third, the algorithm picks a random number g between 2 and $n - 2$, and computes the greatest common divisor (gcd) of g and n . If $\text{gcd}(g, n) \neq 1$, then it happens to be a factor of n and so is returned as the result. Fourth, it’s finally time to actually use the quantum computer (whether it be real, simulated, or replaced by a random number generator). This is the expensive step, and the step that I’m counting in order to compare the different samplers. A quantum circuit based on g and n is generated, and executed, producing a sample m . Fifth, Shor’s algorithm classically computes the fraction that’s closest to $m/4^{\lceil \log_2(n) \rceil}$, limiting the fraction’s denominator d to be at most n . Sixth, a candidate factor is generated by computing $\text{gcd}(n, 1 + g^{\lfloor d/2 \rfloor} \bmod n)$. If the candidate is actually a factor of n , it’s returned as the answer. Otherwise the algorithm restarts.

As an example, let’s consider factoring 253 into 11×23 . 253 isn’t even or prime, so the algorithm advances to picking a random value g in $[2, 251]$. 32 of the 249 possible values of g share a factor with 253, caught by the gcd check, so there’s a 12.9% chance the algorithm terminates due to g before collecting a quantum sample m . Of the 14221312 possible pairs (g, m) , 1752324 (12.3%) of them result in the postprocessing producing a factor. Accounting for both these ways of succeeding, replacing the quantum computer with a random number generator results in nearly a 25% chance of success per quantum sample. So the expected number of shots to factor 253 is a bit more than 4. With a noiseless quantum computer, the expected number of shots to factor 253 would instead be roughly 2.5.

3 Results

The experimental results are shown in Figure 1. Factoring all numbers up to 255 resulted in the simulated noiseless quantum computer being sampled 94 times, the random number generator being sampled 120 times, and the real quantum computer being sampled 121 times.

Not surprisingly, the simulated noiseless quantum computer was the most sample efficient. However, because 255 is such a small number, the advantage isn’t large. There were even times where, if I had stopped the experiment, the noiseless simulator would have lost to the random number generator. For large numbers, Shor’s algorithm has an enormous advantage over random guessing. For these small numbers, the advantage is more tenuous.

Humorously, the real quantum computer does slightly worse than the random number generator. Following the plot from left to right: the real quantum computer has a bad start, then manages to keep pace, and even starts to gain near the end, but falls just short in a photo finish [Mun]. I could repeat the experiment many times to verify this was just random bad luck, but IBM only provides 10 free minutes of quantum computer time per month and I consumed half of that running the experiment once. Also, this way’s funnier.

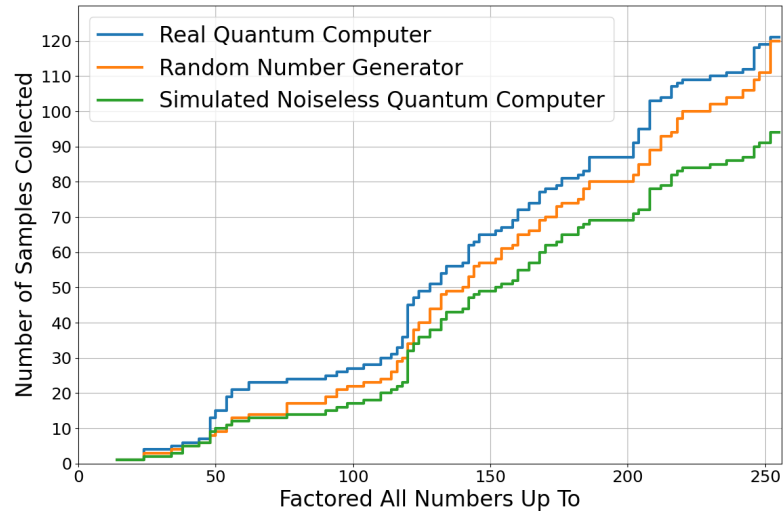


Figure 1: Left: A creative commons sketch of Buzz Lightyear from Deviant Art [jam]. Right: comparing the sample efficiency of factoring all numbers up to 255 with a variety of sampling devices.

4 Conclusion

To my knowledge, no one has cheated at factoring in this way before. Given the shenanigans pulled by past factoring experiments, that’s remarkable.

Ultimately, the key to factoring small numbers isn’t making the quantum computer “work well”. That’s the key to factoring *large* numbers. For small numbers it’s sufficient to strap a hopelessly oversized circuit to the quantum processor, light the fuse, and shout “To infinity and beyond!”.

References

- [Cuc+04] Steven A. Cuccaro, Thomas G. Draper, Samuel A. Kutin, and David Petrie Moulton. *A new quantum ripple-carry addition circuit*. 2004. DOI: [10.48550/ARXIV.QUANT-PH/0410184](https://arxiv.org/abs/quant-ph/0410184). URL: <https://arxiv.org/abs/quant-ph/0410184>.
- [IBM] IBM. *Job limits - IBM Quantum Documentation*. <https://docs.quantum.ibm.com/guides/job-limits>. Accessed: 2024-03-14.
- [jam] jameson9101322. *12 Buzz Lightyear*. <https://www.deviantart.com/jameson9101322/art/12-Buzz-Lightyear-186311087>. Accessed: 2024-03-14.
- [Lan+07] B. P. Lanyon, T. J. Weinhold, N. K. Langford, M. Barbieri, D. F. V. James, A. Gilchrist, and A. G. White. “Experimental Demonstration of a Compiled Version of Shor’s Algorithm with Quantum Entanglement”. In: *Physical Review Letters* 99.25 (Dec. 2007). ISSN: 1079-7114. DOI: [10.1103/physrevlett.99.250505](https://doi.org/10.1103/physrevlett.99.250505). URL: <http://dx.doi.org/10.1103/PhysRevLett.99.250505>.
- [Lu+07] Chao-Yang Lu, Daniel E. Browne, Tao Yang, and Jian-Wei Pan. “Demonstration of a Compiled Version of Shor’s Quantum Factoring Algorithm Using Photonic Qubits”. In: *Physical Review Letters* 99.25 (Dec. 2007). ISSN: 1079-7114. DOI: [10.1103/physrevlett.99.250504](https://doi.org/10.1103/physrevlett.99.250504). URL: <http://dx.doi.org/10.1103/PhysRevLett.99.250504>.
- [Luc+12] Erik Lucero, R. Barends, Y. Chen, J. Kelly, M. Mariantoni, A. Megrant, P. O’Malley, D. Sank, A. Vainsencher, J. Wenner, T. White, Y. Yin, A. N. Cleland, and John M. Martinis. “Computing prime factors with a Josephson phase qubit quantum processor”. In: *Nature Physics* 8.10 (Aug. 2012), pp. 719–723. ISSN: 1745-2481. DOI: [10.1038/nphys2385](https://doi.org/10.1038/nphys2385). URL: <http://dx.doi.org/10.1038/nphys2385>.

- [Mar+12] Enrique Martín-López, Anthony Laing, Thomas Lawson, Roberto Alvarez, Xiao-Qi Zhou, and Jeremy L. O’Brien. “Experimental realization of Shor’s quantum factoring algorithm using qubit recycling”. In: *Nature Photonics* 6.11 (Oct. 2012), pp. 773–776. ISSN: 1749-4893. DOI: [10.1038/nphoton.2012.259](https://doi.org/10.1038/nphoton.2012.259). URL: <http://dx.doi.org/10.1038/nphoton.2012.259>.
- [ME99] Michele Mosca and Artur Ekert. “The Hidden Subgroup Problem and Eigenvalue Estimation on a Quantum Computer”. In: *Quantum Computing and Quantum Communications*. Springer Berlin Heidelberg, 1999, pp. 174–188. ISBN: 9783540492085. DOI: [10.1007/3-540-49208-9_15](https://doi.org/10.1007/3-540-49208-9_15). URL: http://dx.doi.org/10.1007/3-540-49208-9_15.
- [Mun] Randall Munroe. *XKCD - Sports*. <https://xkcd.com/904/>. Accessed: 2024-03-14.
- [SSV13] John A. Smolin, Graeme Smith, and Alexander Vargo. “Oversimplifying quantum factoring”. In: *Nature* 499.7457 (July 2013), pp. 163–165. ISSN: 1476-4687. DOI: [10.1038/nature12290](https://doi.org/10.1038/nature12290). URL: <http://dx.doi.org/10.1038/nature12290>.
- [Van+01] Lieven M. K. Vandersypen, Matthias Steffen, Gregory Breyta, Costantino S. Yannoni, Mark H. Sherwood, and Isaac L. Chuang. “Experimental realization of Shor’s quantum factoring algorithm using nuclear magnetic resonance”. In: *Nature* 414.6866 (Dec. 2001), pp. 883–887. ISSN: 1476-4687. DOI: [10.1038/414883a](https://doi.org/10.1038/414883a). URL: <http://dx.doi.org/10.1038/414883a>.